

Clase 184 — Splunk para detección

Parte: 8 — Blue Team, detección y SOC · Fuente: Blue Team Handbook: SOC, SIEM, and Threat Hunting Use Cases — Don Murdoch 🕒 Duración estimada: 120 min · Nivel: Intermedio

Objetivo

Dominar los fundamentos del lenguaje de búsqueda de Splunk (SPL) aplicado a detección: filtrar, transformar, agregar y correlacionar eventos para construir búsquedas de detección y alertas programadas. Splunk es uno de los SIEM más extendidos; saber consultarlo con soltura es una habilidad central del analista de SOC.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Escribir** búsquedas SPL con filtros, `stats`, `eval` y `where`.
- 2. **Construir** detecciones basadas en umbrales, agregaciones y línea base.
- 3. **Usar** `tstats` y modelos de datos para búsquedas rápidas.
- 4. **Programar** alertas (saved searches) con acciones y throttling.
- 5. **Aplicar** el ciclo Notable Event de Splunk Enterprise Security a nivel conceptual.

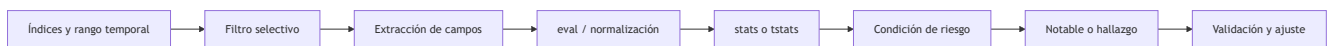
Temas

#	Tema	Por qué importa
1	Anatomía de una búsqueda SPL	Base de todo lo demás
2	Comandos de filtrado y campos	Reduce ruido y aísla lo relevante
3	<code>stats</code> , <code>eval</code> , <code>where</code>	Agregación y lógica de detección
4	<code>tstats</code> y modelos de datos (CIM)	Velocidad a escala
5	Detección por umbral vs anomalía	Distintos tipos de caso de uso
6	Saved searches y alertas	Automatiza la detección
7	Lookups y enriquecimiento	Añade contexto (activos, intel)
8	Notable Events (Splunk ES)	Cómo se gestiona la alerta en producción

Explicación en profundidad

SPL funciona como una tubería: cada comando recibe un conjunto de resultados y produce otro. El orden importa. Filtrar temprano por tiempo, índice, sourcetype y términos selectivos reduce el trabajo posterior; extraer campos innecesarios antes del filtro encarece la consulta. `stats` agrega eventos disponibles,

mientras `tstats` aprovecha estructuras aceleradas y exige comprender el modelo de datos y sus restricciones.



El Common Information Model no modifica mágicamente los datos: define campos y etiquetas que las fuentes deben mapear. Una búsqueda portable necesita verificar que ese mapeo esté completo y fresco. Además, las búsquedas programadas deben considerar retraso de ingesta. Una ejecución cada cinco minutos sobre los últimos cinco minutos puede perder un evento que llegó seis minutos tarde; una ventana solapada exige deduplicar.

El tuning profesional no consiste en añadir exclusiones indefinidamente. Primero se identifica qué hipótesis genera el ruido; luego se incorpora contexto estable —tipo de activo, cuenta de servicio, firma, relación padre-hijo— y se prueban positivos conocidos y actividad benigna. Toda excepción debe tener dueño, razón y fecha de revisión.

Leer SPL como una explicación comprobable

Considera una búsqueda que pretende encontrar muchos fallos seguidos de un éxito. El filtro inicial selecciona autenticaciones y periodo; la extracción identifica usuario, origen y resultado; la agregación cuenta y ordena; la condición expresa el umbral. Si se agrupa solo por usuario, se mezclan dos orígenes; si se agrupa solo por IP, se mezclan cuentas. La elección de claves forma parte de la hipótesis, no es un detalle de sintaxis.

Antes de optimizar con `tstats`, se ejecuta una versión transparente sobre una muestra pequeña y se inspeccionan eventos. Después se compara el resultado acelerado con esa referencia. Si CIM no mapea `src`, `user` o `action` de una fuente, `tstats` puede devolver menos sin mostrar un error evidente. Esta comparación enseña que rendimiento y corrección son problemas separados.

Para búsquedas programadas se anota: frecuencia, ventana, retraso aceptado y clave de deduplicación. Por ejemplo, ejecutar cada cinco minutos sobre los últimos quince absorbe eventos tardíos, pero puede repetir coincidencias; un identificador estable o una supresión temporal controla duplicados. El diagrama representa esta transformación: cada caja debe poder explicar qué filas entraron, cuáles salieron y por qué. Una detección mantenible permite que otro analista reproduzca esa explicación.

Campos, agregaciones y contexto

Una búsqueda exploratoria suele comenzar con eventos crudos y `table` para verificar nombres y valores. Después `eval` construye campos derivados con una razón explícita; `stats` reduce filas y puede borrar detalle necesario para investigar. Antes de agregar se decide qué evidencia conservar, por ejemplo lista de hosts, primer y último tiempo y conteo distinto de destinos. `transaction` puede resultar intuitivo, pero consume recursos y no siempre modela bien sesiones; agrupaciones con claves y tiempo suelen ser más transparentes.

Lookups permiten añadir inventario, cuentas de servicio o listas autorizadas. Ese contexto necesita propietario y actualización. Una lookup obsoleta puede silenciar al usuario que dejó de ser administrador. Las macros evitan duplicación, pero esconden lógica si no se versionan y documentan. El alumno debe poder expandirlas y explicar la búsqueda completa.

Del resultado a una detección operativa

Una consulta que encuentra un ejemplo no está lista para producción. Se ejecuta sobre periodos benignos, se mide cardinalidad, se identifican entidades dominantes y se prueba con un positivo controlado. Luego se define severidad con impacto del activo y confianza de la señal. El notable incluye campos suficientes para que el analista empiece: identidad, host, tiempo, razón y pivotes, no solo el texto de la regla.

El tuning conserva trazabilidad. En vez de excluir `powershell.exe` para una herramienta administrativa, se limita a ruta, firma, cuenta, padre y activo conocidos; se fija vencimiento. Esto enseña una diferencia decisiva: reducir falsos positivos no debe borrar el comportamiento que la hipótesis quería detectar.

Glosario

- **SPL:** lenguaje de búsqueda de Splunk.
- **Sourcetype:** clasificación que guía cómo interpretar eventos.
- **Pipeline:** secuencia de comandos separados por `|`.
- **Transformación:** operación que resume o cambia el conjunto de resultados.
- **CIM:** modelo común de campos y datasets de Splunk.
- **Data model acceleration:** materialización que permite consultas `tstats` más rápidas.
- **Notable:** evento de seguridad promovido para triaje.
- **Ventana solapada:** rango que vuelve a consultar parte del periodo anterior.

Definiciones y características

- **SPL (Search Processing Language):** lenguaje de consulta de Splunk basado en tuberías (`|`). Característica: componible, cada comando transforma el resultado del anterior.
- **stats :** agrega eventos por campos (`count by user`). Característica: pieza central para líneas base y umbrales.
- **tstats :** consulta datos acelerados/indexados sin escanear eventos crudos. Característica: órdenes de magnitud más rápido.
- **CIM (Common Information Model):** esquema de normalización de Splunk. Característica: permite escribir detecciones portables entre fuentes.
- **Lookup:** tabla externa (CSV/KV) para enriquecer (ej. lista de activos críticos). Característica: cruza eventos con contexto de negocio.
- **Saved search / alert:** búsqueda guardada que corre en horario y dispara acciones. Característica: soporta throttling para evitar spam de alertas.
- **Notable Event:** alerta gestionable en Splunk ES con estado, propietario y urgencia. Característica: el objeto de trabajo del analista.

Búsqueda razonada — fallos seguidos de éxito

La intención es identificar una combinación de cuenta y origen con varios fallos y después un éxito. Una primera búsqueda exploratoria conserva detalle:

```
index=auth earliest=-30m (action="failure" OR action="success")
| table _time user src dest action
| sort 0 user src _time
```

Solo después de comprobar campos se construye una agregación. Contar fallos y éxitos en la misma ventana no demuestra que el éxito ocurrió después; para esa condición se conservan tiempos o se usa una

lógica de secuencia adecuada al entorno. Agrupar solo por `user` mezclaría varios orígenes; agrupar solo por `src` mezclaría cuentas. El par `user`, `src`, y a veces `dest`, pertenece a la hipótesis.

La búsqueda de producción añade ventana solapada por retraso de ingesta y una clave de deduplicación. Una lookup identifica cuentas de servicio, pero no las excluye por completo: cambia el contexto y aplica condiciones propias. El notable muestra cuenta, origen, destinos, primer/último evento, conteos y pivote, de modo que el analista no repita la búsqueda inicial.

Antes de migrar a `tstats`, se mapea la fuente al dataset CIM correspondiente y se compara resultado con SPL sobre eventos crudos. Si una fuente no puebla `action`, la búsqueda acelerada pierde casos aunque sea rápida. El ejercicio enseña que optimización viene después de verificar semántica.

✓ Criterio de dominio

El alumno explica cada comando, demuestra un positivo y un negativo, mide retraso y ejecución, y documenta cualquier excepción con vencimiento. Copiar una consulta que devuelve filas sin justificar agrupación, orden y campos no acredita detección.

Herramientas y preparación

- **Splunk Enterprise Trial** o **Splunk Free** en el Windows/Linux de laboratorio.
- Datos de muestra: la app **Boss of the SOC (BOTS)** de Splunk es un dataset público excelente para practicar detección.
- Un **Universal Forwarder** enviando tus Sysmon/Event Logs de la clase 182.
- Opcional: **Splunk Security Essentials** (app gratuita con ejemplos de detección mapeados a ATT&CK).

Practica siempre sobre datos de laboratorio o el dataset BOTS.

Laboratorio guiado — Detecta con SPL

1. **Explora los datos.** Búsqueda base: `index=main sourcetype=XmlWinEventLog EventCode=1 | head 20`
2. **Cuenta procesos raros.** Línea base de procesos por host: `... EventCode=1 | stats count by Computer, Image | sort - count`
3. **Detecta ejecución sospechosa.** Busca intérpretes lanzados por Office: `... EventCode=1 ParentImage="*\\WINWORD.EXE" (Image="*\\powershell.exe" OR Image="*\\cmd.exe") | table _time, Computer, ParentImage, Image, CommandLine`
4. **Fuerza bruta de autenticación.** Con eventos 4625/4624: `... (EventCode=4625 OR EventCode=4624) | stats count(eval(EventCode=4625)) as fails count(eval(EventCode=4624)) as success by Account_Name | where fails>10 AND success>0`
5. **Acelera con tstats.** Reescribe una de las anteriores usando un modelo de datos CIM y `tstats` para comparar tiempos.
6. **Enriquece con lookup.** Crea un CSV `criticos.csv` (host, propietario) y cruza: `... | lookup criticos.csv host OUTPUT propietario`.
7. **Programa la alerta.** Guarda la búsqueda del paso 3 como alerta cada 5 min con throttling por `Computer` de 30 min y acción de registro.

Ejercicios

1. Escribe una búsqueda que liste los 10 dominios DNS más consultados por host.
2. Detecta `regsvr32.exe` o `mshta.exe` con conexión de red saliente.
3. Construye una línea base de horas de login por usuario y marca actividad fuera de horario.
4. Usa `eval` para clasificar la severidad de un evento en alta/media/baja.
5. Convierte una búsqueda lenta en una con `tstats` y mide la diferencia.
6. Crea un lookup de cuentas de servicio y suprime sus falsos positivos.

Reto verificable

Entrega tres búsquedas SPL de detección con su explicación y una de ellas convertida en alerta programada con throttling. **Criterio de aceptación:** al menos una búsqueda detecta correctamente un patrón malicioso simulado en tu laboratorio (p. ej. Office→PowerShell) sin disparar con la actividad benigna de línea base, y la alerta guardada aparece en el listado de saved searches ejecutándose.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Búsqueda tarda minutos	Escaneas eventos crudos; usa <code>tstats</code> /modelos de datos
<code>stats</code> no agrupa como esperas	Campo no extraído o mal nombrado; verifica con <code>\ fieldsummary</code>
Alerta dispara sin parar	Falta throttling; añade supresión por entidad
Resultados vacíos pero hay datos	Rango de tiempo o índice equivocado; revisa el time picker
CommandLine truncado	Límite de campo; ajusta <code>maxchars</code> /truncation en props.conf

Preguntas frecuentes

 ¿`stats` o `transaction`? Prefiere `stats`: es mucho más eficiente. `transaction` solo cuando necesitas agrupar eventos por sesión con lógica de inicio/fin que `stats` no expresa bien.

 ¿Necesito Splunk ES para detectar? No para aprender. Con Splunk core y saved searches ya construyes detecciones. ES aporta gestión de notables, correlación empaquetada y flujo de incidentes en producción.

 ¿Por qué usar el CIM? Porque una detección escrita contra campos CIM funciona sin importar el fabricante del log, evitando reescribir por cada fuente nueva.

Referencias verificables y alcance

- Splunk SPL Search Reference: documentación oficial de la sintaxis y semántica de los comandos utilizados en los ejemplos — <https://help.splunk.com/en/splunk-enterprise/search/spl-search-reference/10.0/search-commands/tstats>
- Splunk Common Information Model: documentación oficial del modelo de datos usado por `tstats`; una búsqueda CIM depende de que la fuente esté normalizada y el modelo acelerado según el despliegue — <https://help.splunk.com/en/splunk-enterprise/common-information-model/8.5/introduction/overview-of-the-splunk-common-information-model>

- Splunk, supresión de alertas: documentación oficial para evitar notificaciones repetidas dentro de una ventana definida — <https://help.splunk.com/en/splunk-enterprise/alert-and-respond/alerting-manual/10.0/manage-alert-trigger-conditions-and-throttling/throttle-alerts>
- Splunk BOTS v3: repositorio público de Splunk con datos de práctica; sirve como evidencia reproducible de laboratorio, no como fuente normativa — <https://github.com/splunk/botsv3>
- Splunk Security Essentials: aplicación oficial/comunitaria de contenido para explorar casos de uso; cada ejemplo debe validarse contra los propios campos y datos — <https://splunkbase.splunk.com/app/3435>



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Clase anterior

[Clase 183](#) — SIEM: arquitectura y componentes



Siguiente clase

[Clase 185](#) — Elastic Stack y Wazuh